第 1 章绪论

1. 类和对象之间的对应关系？模板类、类、对象之间呢？函数模板和模板函数之间呢？
2. 面向对象基本特征：抽象（归纳）、封装（信息隐藏，public、protected、private）、继承（重用）、多态（重载和虚函数）

第 2 章_C++对 C 的扩展

3. 结构体（默认 public）和类（默认 private）默认属性访问不同，其他基本一样
   struct A{int x}; A a; a.x;
   class A{int x}; A a; a.x; (╳)
4. 内联函数的关键字 inline，声明的时候不可省略 inline，定义的时候可以省略 inline；如果同时声明和定义函数不可省略 inline（联想虚函数的 virtual 关键字、友元函数的 friend 关键字、静态成员的 static 关键字）

inline void fun(); **//声明，inline 不可省略** void main(){fun();} inline void fun(){…} **//定义，inline 可省略**	inline void fun(){…}; **// 同时声明、定义 inline 不可省略** void main(){fun();}

5. &既能表示引用，也能表示取变量的地址

引用：&放在类型和变量之间（&前面有数据类型，后面有变量）
int a=1;
int & b=a;
	取地址：&放在变量之前（&前面没有数据类型，&后面有变量）
	int a;
	int *p=&a;

6. **定义**引用&一定要初始化

int a; int & b; (╳) b = a;	int a; int & b=a;

7. 定义 const 所修饰的对象时要初始化（类比“引用”）
   const float pi=3.1415926f;
   void fun(const int x){…} //函数不修改参数 x
   void A::fun(const int x)const 的数据成员
   //第一个 const 说明不修改函数 fun 的参数 x，第二个 const 说明成员函数 fun 不修改类 A（常成员函数）
   {…}
8. const 修饰指针/变量: const 修饰其前的变量或指针，若 const 前没有变量或指针，则修饰其后的变量或指针
   (1)指向常量的指针：指针把它指向的对象视为常量
   int a,b; const int *p; p=&a; *p=1; (╳) a=1; p=&b; *p=2; (╳) b=2;
   (2)指针常量（常指针）：指针本身为常量，指针指向的地址不能改变:
   int a,b; int * const p=&a（必须初始化）; p=&b;(╳) *p=1; a=1;
   (3)指向常量的指针常量：指针本身为常量，指针把它指向的对象也视为常量：
   int a,b; const int * const p=&a（必须初始化）; p=&b; (╳) *p=1; (╳)
9. 函数重载：参数类型、个数、数目不同都能重载，返回值类型不同不能构成重载
   普通函数重载

如下构成重载	如下不构成重载：
参数个数不同 void fun1(){…}; void fun1(int x){…} void fun1(int x, int y){…}	返回类型不同 void fun1(float x, int y){…} float fun1(float x, int y){…}

<table>
<tr><td>参数类型不同
void fun1(int x, int y){…}
void fun1(int x, float y){…}</td><td colspan="2">仅仅形参名称不同
void fun1(float x, int y){…}
void fun1(float y, int x){…}</td></tr>
<tr><td rowspan="2">参数顺序不同
void fun1(float x, int y){…}
void fun1(int x, float y){…}</td><td colspan="2">函数所在范围不同</td></tr>
<tr><td>void fun1(int x){…}//全局函数
class A{
public:
void fun1(float x){…}//成员函数
};</td><td>class A{
public:
void fun1(int x){ count<<"A"<<endl; }//A 的成员函数
};
class B: public A{
public:
void fun1(float x){ count<<"B"<<endl;}//B 的成员函数
}
void main(){
A a; a.fun1(1); // 输出 A
B b; b.fun1(1); // 输出 B，会进行强制类型转换 1->1.0f
}</td></tr>
</table>

10. 带默认参数的函数，默认参数要靠后，不能靠前或者在中间

void fun(int x, float y = 1.0f, int z=0){
}
fun(1);
fun(1,1.0);
fun(1,1.0,1);
	错误的默认参数
	void fun(int x=0, float y = 1.0f, int z){
	}
		错误的默认参数
		void fun(int x, float y = 1.0f, int z){
		}

11. new 和 delete

new 创建变量/对象（如果是对象会调用构造函数）

delete 销毁变量/对象（如果是对象会调用析构函数）

<table>
<tr><td>int *p=new int; delete p;
int *q=new int(1); delete q;
int *s=new int[10]; delete [] s;
int *t=new int[10](1); (╳)</td><td>class A{
public:
A(int x=0){}
};
int *p=new A; delete p; //1 次构造函数+1 次析构函数
int *q=new A(1); delete q; //1 次构造函数+1 次析构函数
int *s=new A[10]; delete [] s; //10 次构造函数+10 次析构函数</td></tr>
</table>

第 3 章_类与对象

12. 对象和类的关系：数据类型和变量的关系，如果 A 是类， A a,b,c,d[2];

int n=10; A e[n]; (╳)	const int n; n=10; (╳) A e[n];	const int n=10; A e[n];

13. 访问权限 public/protected/protected:

<table>
<tr><td>公有：
class A{
public:
int x;
void fun(){ x=0; }
};
void main(){</td><td>保护：
class A{
protected:
int x;
void fun(){ x=0; }
};
void main(){</td><td>私有：
class A{
int x; //默认 private
private:
void fun(){ x=0; }
};
void main(){</td></tr>
</table>

A a;
a.x=0;
a.fun();
}
	A a;
	a.x=0; (×)
	a.fun();(×)
	}
		A a;
		a.x=0; (×)
		a.fun();(×)
		}

14. 构造函数的执行时间、次数和顺序

调用时间和次数：
A a,b; //调用两次构造函数
A c[4]; //调用 4 次构造函数
A *p; //不调用构造函数
A *p=new A;//调用 1 次构造函数
A *p=new A[5];//调用 5 次构造函数
	调用顺序：
	class A1{
	public: A1(){ cout<<"A1 的构造函数"<<endl; } };
	class B{
	public: B(){ cout<<"B 的构造函数"<<endl; } };
	class A2: public A1{
	public: A2(){ cout<<"A2 的构造函数"<<endl; }
	B b;
	};
	void main(){
	A2 a; //构造函数调用顺序 A1(祖先)->B(客人)->A2(自己)
	}

15. 构造函数的参数(重载)

构造函数的名字是“类名”

构造函数没有返回类型和返回值

用户没定义构造函数，系统提供默认无参数构造函数

用户定义了构造函数（无论是否有参数），系统不再提供默认的无参数构造函数；如需无参构造函数，请自己重载定义

class A{
};
void main(){
A a; //调用系统提供的默认无参构造函数
}
	class A{
	public:
	A(int x){…} //自定义有参数构造函数
	};
	void main(){
	A a; (×)//没有无参构造函数
	}

class A{
public:
A(){…} //自定义无参构造函数
A(int x){… //自定义有参构造函数
};
void main(){
A a;//调用自定义无参构造函数
A b(2);//调用自定义有参构造函数
}
	class A{
	public:
	A(int x=0){… //自定义有参带默认值构造函数
	};
	void main(){
	A a;//调用自定义有参(用默认值 0)构造函数
	A b(2);//调用自定义有参(参数为 2)构造函数
	}

16. 析构函数的名字为“~类名”

析构函数没有返回值和返回类型

析构函数没有参数，所以不能重载

如果用户没有自定义析构函数，系统提供默认析构函数，什么都不做

class A1{
public: ~A1(){ cout<<"A1 的析构函数"<<endl; } };
class B{
public: ~B(){ cout<<"B 的析构函数"<<endl; } };
class A2: public A1{
public: ~A2(){ cout<<"A2 的析构函数"<<endl; }
	class A{
	};
	void fun(){
	A a1;
	A *pa=new A;//调用构造函数
	…//其他代码

B b;
};
void main(){
A2 a;
} //析构函数调用顺序 A2(自己)-> B(客人)->A1(祖先)
	delete pa;//调用对象*pa 的析构函数(系统提供的默认析构函数)
	}// 调用对象 a1 的析构函数(系统提供的默认析构函数)

17. 系统提供的默认函数：无参构造函数、析构函数、赋值函数“=”、拷贝构造函数
默认构造函数：什么都不做，需要对特定数据初始化时请自定义构造函数
默认析构函数：什么都不做，需要清理数据或分配的内存时请自定义析构函数，尤其是 delete 之前用 new 分配的内存
默认的赋值=：浅拷贝，按值赋值，有指针成员请自定义赋值=
默认的拷贝构造函数：浅拷贝，按值拷贝，有指针成员请自定义拷贝构造函数
18. 拷贝构造函数也是构造函数，是重载了参数为“当前类对象引用”的构造函数，无返回值和返回类型
赋值运算符=本质上上函数“operator=”

class A{
private:
int m_data;
}
void main(){
A a1; //调用系统提供的默认构造函数
A a2(a1); //调用默认拷贝构造函数，此时 a2 正在创建
A a3=a1;//调用默认拷贝构造函数，此时 a3 正在创建
a1=a2; //调用默认赋值=运算符，此时 a1 已经存在
}
	class A{
	private:
	char *m_pData;
	int m_cnt;
	public:
	A(int cnt){//自定义构造函数
	m_cnt=cnt;
	m_pData=new int[m_cnt];
	}
	A(A &a){ //自定义拷贝构造函数
	m_cnt = a.m_cnt;
	m_pData=new int[m_cnt];
	strcpy(m_pData, a.m_pData, m_cnt);
	A(A* pA)(){…}//重载的构造函数，但不是拷贝构造函数
	A& operator=(A& a){//自定义赋值运算符=
	if (this==&a) return *this;
	delete [] m_pData;
	m_cnt = a.m_cnt;
	m_pData=new int[m_cnt];
	strcpy(m_pData, a.m_pData, m_cnt);
	return *this;
	}
	~A(){//自定义析构函数
	delete [] m_pData;
	}
	}

19. 拷贝构造函数的调用时间、向函数传递对象/对象引用/对象地址

用老对象初始化新对象	对象作为函数参数	对象作为函数返回值

class A{…};
A a1; //构造函数
A a2(a1);//拷贝构造函数
A a3(a2);//拷贝构造函数
A a4=a3;//拷贝构造函
	class A{…};
	void fun1(A a){…}
	void fun2(A& a){…}
	void fun3(A* pa){…}
	void main(){
	A a1;//构造函数
	fun1(a1);//传对象，调用拷贝构造函数
		class A(){…}
		A fun1(){ A a; return a; }
		A& fun2(A &a){ return a; }
		A* fun3(){ A* pa=new A; return pa; }
		void main(){
		fun1(); // 调用 1 次构造函数+1 次拷贝构造函数

数
a4=a3;//赋值函数=
	fun2(a1);//传对象引用，不调用拷贝构造函数
	fun3(&a1); //传对象地址，不调用拷贝构造函数
	}
		A a1； // 调用 1 次构造函数
		A a2(a1); //调用 1 次拷贝构造函数
		A a3=fun1(); //调用 1 次构造函数+2 次拷贝构造函数
		fun2(a1);//调用 0 次拷贝构造函数
		A a4=fun2(a1); //调用 1 次拷贝构造函数
		a4=fun2(a1);//调用 0 次拷贝构造函数
		fun3(&a1);//调用 1 次构造函数+0 次拷贝构造函数
		A* pa=fun3(&a1);//调用 1 次构造函数+0 次拷贝构造函数
		}

20. 类的普通成员函数(相对于静态成员函数而言)都有 this 指针，包括构造函数、析构函数、拷贝构造函数、重载运算符等

21. 静态数据成员不单属于某个对象，而是属于整个类
对象可以访问静态数据成员（public 类型才行），也可以通过类和域作用符直接访问
静态数据成员必须在类外进行初始化

class A{
public:
A(int x=0){ cnt++; m_x=x; }
static int cnt;
int m_x;
};
int A::cnt=0;
void main(){
A a;
cout<<a.m_x<<a.cnt<<A::cnt; //输出 0、1、1
A aa(10);
cout<<aa.m_x<<aa.cnt<<A::cnt; //输出 10、2、2
}
	class A{
	static int num;
	public:
	static int cnt=0; (╳)
	int x=0; (╳)
	};
	int A::num=0;
	void main(){
	A a,b;
	cout<<a.x<<b.x;
	cout<<A::x; (╳)x 不是静态成员
	A::num=1; (╳) 私有
	a.num=2; (╳)私有
	}

22. 静态成员函数(包括其他成员函数)都属于整个类
静态成员函数只能访问类的静态数据成员，若访问非静态数据成员必须将对象作为参数传入
普通成员函数可以访问类的静态数据成员，也可以方位非静态数据成员

```
class A{
private:
   int x;
   static int y;
public:
void fun1(){
     x=0;y=1; //相当于 this->x=0; this->y=1;
   }
   static void fun2(){
     x=0; (╳)//静态成员函数不能访问非静态数据，因为静态成员函数中没有 this 指针
     y=1;
   }
```

```
    static void fun3(A &a){
    a.x=0;
      a.y=1;
      x=0; (╳)//静态成员函数不能访问非静态数据，因为静态成员函数中没有 this 指针
      y=1;
    };
};
int A::y=0;
void main(){
    A a;
    a.fun1();
    a.fun2();
    a.fun3(a);
    A::fun1();(╳)
    A::fun2();
    A::fun3(a);
}
```

23. 友元函数内可以访问类内私有和保护成员

```
class A{
  private:
    int x;
    friend void fun1(A &a);
};
void fun1(A &a){
a.x=0;
}
void fun2(A &a){
    a.x=0; (╳)//普通函数中不能通过对象直接访问 private/protected 数据
}
```

第 4 章_派生类与继承

24. 派生权限 public/protected/private

继承方式 / 基类成员	公有继承 public	保护继承 protected	私有继承 private
公有成员 public	public	protected	private
保护成员 protected	protected	protected	Private
私有成员 private	不能继承	不能继承	不能继承

不同继承方式的影响主要体现在：

- ✓ 派生类**成员函数**对基类成员的访问权限
- ✓ 通过派生类**对象**对基类成员的访问权限

class A	class A	class A

```
{ public:
    A()   {i1 = 0; j1 = 0;}
    void f1(){
      i1=1; j1=1;
    }
  protected:     int j1;
  private:     int i1;
};
class B:public A
{ public:
      B() {i2 = 0; j2 = 0;}
      void   f2(){
        i1=2; (×)   j1=2;
        i2=2;   j2=2;
      }
  protected:    int j2;
  private:       int i2;
};
void main(){
A a;
a.i1=3; (×)   a.j1=3; (×)   a.f1();
B b;
b.i1=3; (×) b.j1=3; (×) b.f1();
b.i2=4; (×) b.j2=4; (×) b.f2();
}
```

```
{ public:
    A()   {i1 = 0; j1 = 0;}
    void f1(){
      i1=1; j1=1;
    }
  protected:     int j1;
  private:     int i1;
};
class B:protected A
{ public:
      B() {i2 = 0; j2 = 0;}
      void   f2(){
        i1=2; (×)   j1=2;
        i2=2;   j2=2;
      }
  protected:    int j2;
  private:       int i2;
};
void main(){
A a;
a.i1=3; (×)   a.j1=3; (×)   a.f1();
B b;
b.i1=3; (×) b.j1=3; (×) b.f1();(×)
b.i2=4; (×) b.j2=4; (×) b.f2();
}
```

```
{ public:
    A()   {i1 = 0; j1 = 0;}
    void f1(){
      i1=1; j1=1;
    }
  protected:     int j1;
  private:     int i1;
};
class B:private A
{ public:
      B() {i2 = 0; j2 = 0;}
      void   f2(){
        i1=2; (×)   j1=2;
        i2=2;   j2=2;
      }
  protected:    int j2;
  private:       int i2;
};
void main(){
A a;
a.i1=3; (×)   a.j1=3; (×)   a.f1();
B b;
b.i1=3; (×) b.j1=3; (×) b.f1();(×)
b.i2=4; (×) b.j2=4; (×) b.f2();
}
```

25. 构造函数和析构函数的调用顺序

构造顺序：祖先→客人→自己

析构函数：自己→客人→祖先

```
classA1{
public:
  A1(){cout<<"A1"<<endl;}
  ~A1(){cout<<"~A1"<<endl;}
};
class B{
public:
  B(){cout<<"B"<<endl;}
  ~B(){cout<<"~B"<<endl;}
};
class A2: public A1{
public:
  B m_b;
  A2(){cout<<"A2"<<endl;}
  ~A2(){cout<<"~A2"<<endl;}
};
void main(){
  A2 a; //输出 A1、B、A2
  A2 *pa = new A2; //输出 A1、B、A2
  delete pa; //输出（析构 pa 所指对象）：~A2、~B、~A1
} //输出（析构 a）：~A2、~B、~A1
```

26. 赋值兼容性

派生类的对象/引用/指针，可以赋值给基类的对象/引用/指针

```
class A{
public:
   void fun1(){ … }
};
class B: public A{
public:
   void fun2(){ … }
}
void main(){
   A a;
   B b;

   a=b; //调用的是 A 的默认赋值运算符“=”
   A* pa=&b;
   A & ra=b;

   a.fun1();
   a.fun2();(╳)//通过基类的对象只能访问基类中有的函数
   pa->fun1();
   pa->fun2();(╳) //通过基类的指针只能访问基类中有的函数
   ra.fun1();
   ra.fun2();(╳) //通过基类的引用只能访问基类中有的函数
   b.fun1();
   b.fun2();
}
```

27. 构造函数和拷贝构造函数的初始化表，

“：”**显示**调用基类的构造函数

+**显示**调用对象数据成员的构造函数

+初始化 const 数据成员

+初始化引用数据成员

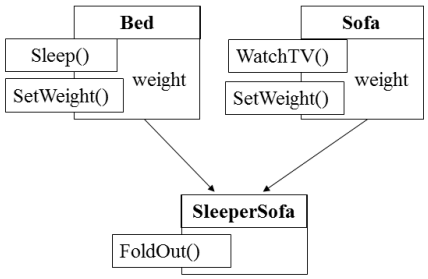
+初始化普通数据成员（普通数据成员也可以放在构造函数体内{…}）

显示调用基类的构造函数
+**显示**调用对象数据成员的构造函数
+初始化 const 数据成员
+初始化引用数据成员
class A{
public:
A(int x){ m_x=x; }
protected:
int m_x;
};
class B: public A{
public:
B(){}//错误，无法调用父类构造函数和成员对象构造函数，因为他们需要参数
B(int &i) :A(i),m_a(i), ten(10),refi(i) {…}
protected:
	子类的自定义构造函数会调用父类的构造函数和对象成员的构造函数，至于调用哪一个需要根据子类的构造函数初始化表传的参数
	class A{
	public:
	A(){cout<< ″A1 ″<<endl;}
	A(A& a){ cout<< ″A2 ″<<endl;}
	A(int x){cout<< ″ A3″<<endl;}
	};
	class B: public A{
	public:
	B(){cout<< ″B1 ″<<endl;}
	B(B & b){cout<< ″B2 ″<<endl;}
	B(int x) {cout<< ″B3 ″<<endl;}
	B(float x):A(1) {cout<< ″B4 ″<<endl;}
	B(char x) :A(1),m_a(1){cout<< ″B5

```
    A m_a;
    const int ten; //常量数据成员
    int &refi; //引用数据成员
};
void main(){
  int x = 0;
  B b1(x);
}
```

```
"<<endl;}
protected:
    A m_a;
};
void main(){
  B b1;//输出 A1、A1、B1
  B b2(b1); //输出 A1、A1、B2
  B b3(1); //输出 A1、A1、B3
  B b4(1.0f); //输出 A3、A1、B4
  B b5('a'); //输出 A3、A3、B5
}
```

28. 多继承的二义性：同名属性会出现多个继承副本，调用产生歧义，因此需要借助虚继承来解决

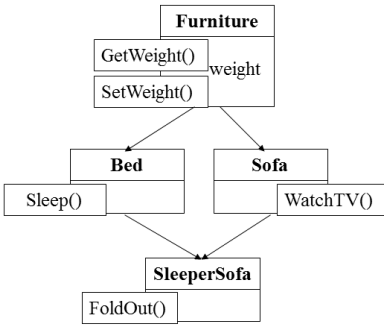



```
SleeperSofa ssofa;
ssofa.weight; // 二义性
ssofa.SetWeight(); // 二义性
ssofa.Bed::weight; // ok
ssofa.Bed::SetWeight(); // ok
```

✓解决二义性问题的根本办法是引入虚继承的概念

✓虚继承：在建立继承关系时，加virtual关键字，C++编译器能自动分辨这种二义性。

✓派生类的对象在构造时对虚基类的子对象只构造一次——有效解决二义性问题。




```
class Bed :virtual public Furniture    {…};
class Sofa :virtual public Furniture   {…};
class SleeperSofa :public Bed , public Sofa
    {…};
```

38

第 5 章_多态性

29. 运算符重载，本质上是函数重载
    运算符可以重载为成员函数、友元函数

不能重载的运算符：	只能重载为成员函数：
:: 作用域限定符 ? : 条件运算符 . 访问对象成员 * 通过指针间接访问变量 # 编译器预处理命令的开始符号	=、[]、->、()

30. ++、--要注意前置和后置的区别
    int a=1;

int b=a++;运行结束后，a 的值为 2，b 的值为 1	int b=++a;//运行结束后，a 的值为 2，b 的值也是 2

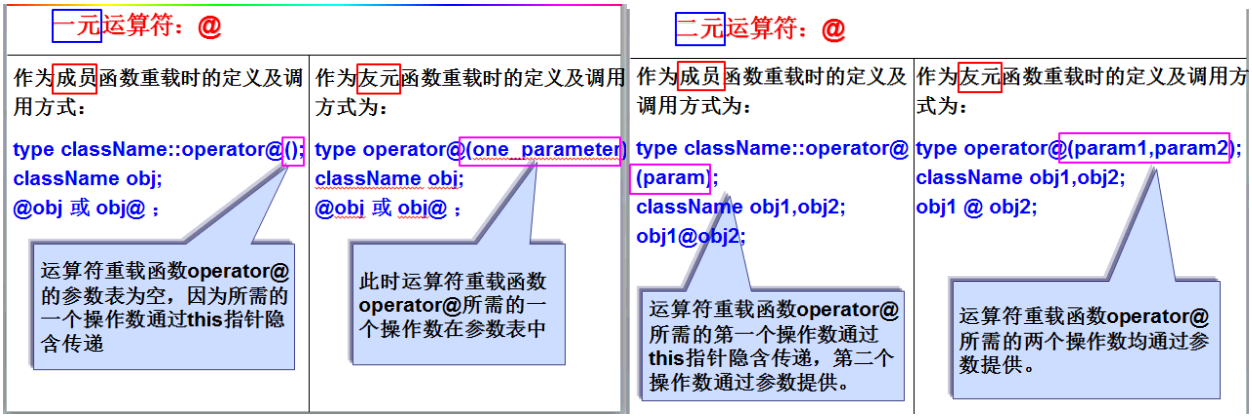
成员函数：	友元函数：
class A{ public: A(float x){m_x=x;} private: float m_x;	class A{ public: A(float x){m_x=x;} private: float m_x;

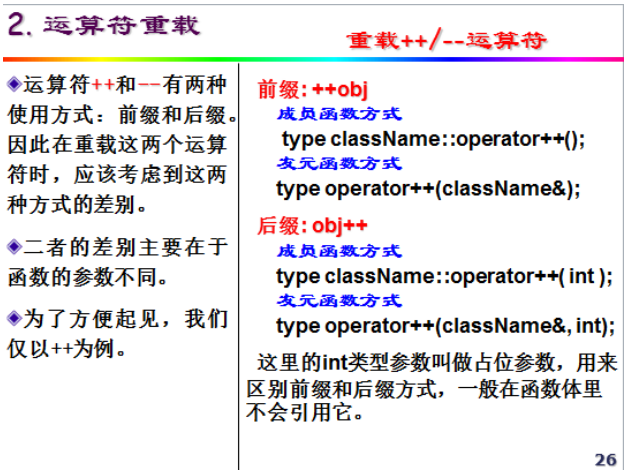
A operator+(A); //成员函数声明
};
A A::operator+(A a)　//成员函数定义
{ A t;
t.m_x = m_x + a.m_x;
return t;
}
void main( )
{ A a1(1), a2(3);
A a3=a1+a2;//OK
A a4 =a1 + 27;　//OK，相当于 a4=a1.operator+(A(27))
A a5 = 27+ a1;　//(╳)相当于 a5=27.operator+(a1)
}
	friend A operator+(A, A); //友元函数声明
	};
	A operator+(A a1, A a2)　//友元函数定义
	{ A t;
	t.m_x = a1.m_x + a2.m_x;
	return t;
	}
	void main( )
	{ Aa1(1), a2(3);
	A a3=a1+a2;//OK
	A a4 =a1 + 27;　//OK，相当于 a4=operator+(a1,A(27))
	A a5 = 27+ a1;　//OK，相当于 a5=operator+(A(27), a1)
	}

31. 单目、双目运算符

单目运算符重载为成员不需要参数（唯一一个参数通过本身 this 指针），单目运算符重载为友元需要提供一个参数

双目运算符重载为成员只需要一个参数（第一个参数通过本身 this 指针），双目运算符重载为友元需要两个参数





32. 只有类的成员函数才能声明为虚函数 virtual 关键字

静态成员函数不能声明为虚函数

内联函数不能声明为虚函数

构造函数不能声明为虚函数

析构函数通常声明为虚函数

子类从父类继承下来的虚函数可以省略 virtual

33. 函数重载在编译阶段，virtual 虚函数调用在运行阶段(通过引用或指针调用)

```
class A{
public:
  void f1(){cout<<"A::f1"<<endl;}
  virtual void f2(){cout<<"A::f2"<<endl;}
  virtual void f3(){cout<<"A::f3"<<endl;}
```

```
}
class B: public A{
public:
  void f1(){cout<<"B::f1"<<endl;}
  void f2(){cout<<"B::f2"<<endl;}
  void f3(int x){ cout<<"B::f3"<<endl;}
}
```

void main{
B b;
A a = b;
a.f1();//输出 A::f1
a.f2();//输出 A::f2
a.f3();//输出 A::f3
a.f3(1);// (×)
}
	void main{
	B b;
	A &a = b;
	a.f1();//输出 A::f1
	a.f2();//输出 B::f2
	a.f3();//输出 A::f3
	a.f3(1);// (×)
	}
		void main{
		B b;
		A *pa = &b;
		pa->f1();//输出 A::f1
		pa->f2();//输出 B::f2
		a.f3();//输出 A::f3
		a.f3(1);// (×)
		}

void f(A a){
a.f1();
a.f2();
a.f3();
a.f3(1);// (×)
}
void main(){
A a;
B b;
f(a);//输出 A::f1、A::f2、A::f3
f(b); //输出 A::f1、A::f2、A::f3
}
	void f(A &a){
	a.f1();
	a.f2();
	a.f3();
	a.f3(1);// (×)
	}
	void main(){
	A a;
	B b;
	f(a);//输出 A::f1、A::f2、A::f3
	f(b); //输出 A::f1、B::f2、A::f3
	}
		void f(A *pa){
		pa->f1();
		pa->f2();
		pa->f3();
		pa->f3(1);// (×)
		}
		void main(){
		A a;
		B b;
		f(&a);//输出 A::f1、A::f2、A::f3
		f(&b); //输出 A::f1、B::f2、A::f3
		}

34. virtual 析构函数通常声明为虚函数

class A{
public:
~A(){cout<<"~A"<<endl;}
}
class B: public A{
public:
~B(){cout<<"~B"<<endl;}
}
void main(){
A *pa = new A;
delete pa;//输出~A
B *pb = new B;
delete pb;//输出~B 和~A
pa = new B;
delete pa;//输出~A
}
	class A{
	public:
	virtual ~A(){cout<<"~A"<<endl;}
	}
	class B: public A{
	public:
	~B(){cout<<"~B"<<endl;}
	}
	void main(){
	A *pa = new A;
	delete pa;//输出~A
	B *pb = new B;
	delete pb;//输出~B 和~A
	pa = new B;
	delete pa;//输出~B 和~A
	}

35. 虚函数关键字 virtual

纯虚函数=0→抽象类不能定义对象

```
class A{
  public:
    virtual void fun()=0;//纯虚函数
    virtual ~A()=0;//纯虚析构函数
```

```
};
class B: public A{
};
class C: public A{
public:
  virtual void fun(){ … } // virtual 可以省略
};
class D: public B{
   public:
virtual void fun(){ … } // virtual 可以省略
};
void main(){
  A a;(╳) //A 是抽象类不能定义对象
  B b;(╳) //B 也是抽象类不能定义对象
  C c;
  D d;
}
```

第 6 章_模板

36. 函数模板/模板函数

单个模板参数	多个模板参数

//函数模板
template <typename T>
T max(T a,T b)
{ return a>b?a:b;}
void main()
{
floatc = **max(1.0f, 1.5f)**; //模板函数
float d = max(1, 1.5f); //错误，无法匹配
}
	//函数模板
	template <typename T, typename S>
	T max(T a, S b)
	{ return (T)(a>b?a:b);}
	void main()
	{
	//模板函数
	float c = max(1.5f, 1); //结果 **1.5f**
	int d = max(1, 1.5f); // 结果为 **1**
	float d = max(1, 1.5f); // 结果为 **1.0f**
	}

37. 类模板/模板类/对象

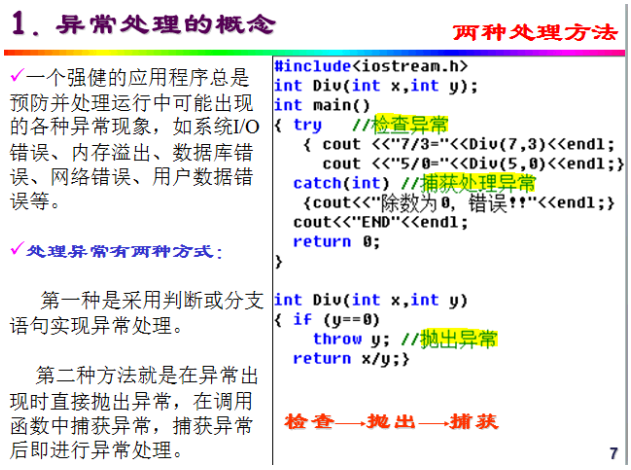
单个模板参数	多个模板参数

template <class T>//类模板声明
class pair {// 类名
public:
pair(T first, T second) //类成员函数
{
Value1=first;
Value2=second;}
private:
T value1,value2; //数据成员
};
 	
template <class **T**>
	template <class T, class S>//类模板声明
	class pair {// 类名
	public:
	pair(T first, S second)； //类成员函数
	{ Value1=first;
	Value2=second;}
	private:
	T value1;//数据成员
	S value2; //数据成员
	};
	void main()
	{

pair<T,>::pair(T first, T second) //在类模板外定义类成员函数
{
Value1=first;
Value2=second;
}
 	
void main(){
pair<int> obj1(1,2); //生成模板类并创建对象：
}
	pair<int, float>obj1(1, 2.0f); //生成模板类并创建对象：
	}

第 7 章_异常处理

38. 异常处理步骤



教材：

3.3 构造函数和析构函数

3.4 类的聚集—对象成员

3.5 静态成员

5.1 单一继承

5.4 类模版

6.1 运算符重载

6.2 虚函数

6.3 纯虚函数和抽象类

2 套模拟试题